VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum- 590014, Karnataka.

LAB RECORD

on

Big Data Analytics (23CS6PCBDA)

Submitted by

Vinayak Sunil Rodd (1WA23CS045)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU - 560019
January 2026 – May 2026

B.M.S. College of Engineering

Bull Temple Road, Bangalore 560019
(Affiliated to Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Big Data Analytics" carried out by **Vinayak Sunil Rodd (1WA23CS045),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfilment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum during the year 2026. The Lab report has been approved as it satisfies the academic requirements in respect of a Big Data Analytics (23CS6PCBDA) work prescribed for the said degree.

Anita Harish Kenchannavar
Assistant Professor
Department of CSE, BMSCE

Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

INDEX

Sl. No.	Date	Experiment Title	Page No.
1	30.01.26	MongoDB- CRUD Operations Demonstration (Practice and Self Study)	5

2	06.02.26	Perform the following DB operations using Cassandra.
		• Create a keyspace by name Employee
		• Create a column family by name
		  o Employee-Info with attributes
		  o Emp_Id Primary Key, Emp_Name, Designation,
		  o Date_of_Joining, Salary, Dept_Name
		• Insert the values into the table in batch
		• Update Employee name and Department of EmpId 121
		• Sort the details of Employee records based on salary
		• Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee.
		• Update the altered table to add project names.
		• Create a TTL of 15 seconds to display the values of Employees.
			8

3	13.02.26	Perform the following DB operations using Cassandra.
		• Create a keyspace by name Library
		• Create a column family by name Library-Info with attributes
		  o Stud_Id Primary Key,
		  o Counter_value of type Counter,
		  o Stud_Name, Book-Name, Book-Id,
		  o Date_of_issue
		• Insert the values into the table in batch
		• Display the details of the table created and increase the value of the counter
		• Write a query to show that a student with id 112 has taken a book “BDA” 2 times.
		• Export the created column to a csv file
		• g) Import a given csv dataset from local file system into Cassandra column family
			10

4	27.02.26	Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)	12
5	06.03.26	Implement Wordcount program on Hadoop framework	13

6	13.03.26	From the following link extract the weather data https://github.com/tomwhite/hadoop-book/tree/master/input/ncdc/all • Create a MapReduce program to find average temperature for each year from NCDC data set. • b) find the mean max temperature for every month	17

7	10.04.26	For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.	22
8	10.04.26	Write a Scala program to print numbers from 1 to 100 using for loop.	29
9	10.04.26	Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.	30
10	2.04.26	Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).	31

Github Link: https://github.com/Vinayak1205/BDALab/tree/main

Course Outcomes (COs):

CO1	Apply the concept of NoSQL, Hadoop or Spark for a given task
CO2	Analyse big data analytics mechanisms that can be applied to obtain solution for a given problem.
CO3	Design and implement solutions using data analytics mechanisms for a given problem.

LABORATORY PROGRAM – 1

MongoDB- CRUD Operations Demonstration (Practice and Self Study)

COMMAND WITH OUTPUT - USING ATLAS

```
test> use BDA_LAB
switched to db BDA_LAB
BDA_LAB> db.createCollection("Employee")
{ ok: 1 }
BDA_LAB> db.Employee.insertOne({
...     Emp_Id: 101,
...     Name: "Manoj",
...     Dept: "CSE",
...     Salary: 50000
... })
{
  acknowledged: true,
  insertedId: ObjectId('69f5bb2311d23b4927149f48')
}
BDA_LAB> db.Employee.insertMany([
...     {Emp_Id: 102, Name: "Rahul", Dept: "ISE", Salary: 40000},
...     {Emp_Id: 103, Name: "Anjali", Dept: "ECE", Salary: 45000},
...     {Emp_Id: 104, Name: "Kiran", Dept: "CSE", Salary: 55000}
... ])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69f5bb2a11d23b4927149f49'),
    '1': ObjectId('69f5bb2a11d23b4927149f4a'),
    '2': ObjectId('69f5bb2a11d23b4927149f4b')
  }
}
BDA_LAB> db.Employee.find()
[
  {
    _id: ObjectId('69f5bb2311d23b4927149f48'),
    Emp_Id: 101,
    Name: 'Manoj',
```

```
  }
}
BDA_LAB> db.Employee.find()
[
  {
    _id: ObjectId('69f5bb2311d23b4927149f48'),
    Emp_Id: 101,
    Name: 'Manoj',
    Dept: 'CSE',
    Salary: 50000
  },
  {
    _id: ObjectId('69f5bb2a11d23b4927149f49'),
    Emp_Id: 102,
    Name: 'Rahul',
    Dept: 'ISE',
    Salary: 40000
  },
  {
    _id: ObjectId('69f5bb2a11d23b4927149f4a'),
    Emp_Id: 103,
    Name: 'Anjali',
    Dept: 'ECE',
    Salary: 45000
  },
  {
    _id: ObjectId('69f5bb2a11d23b4927149f4b'),
    Emp_Id: 104,
    Name: 'Kiran',
    Dept: 'CSE',
    Salary: 55000
  }
]
BDA_LAB>
```

```
BDA_LAB> db.Employee.updateOne(
...      {Emp_Id: 101},
...      {$set: {Salary: 60000}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
BDA_LAB> db.Employee.updateMany(
...      {Dept: "CSE"},
...      {$set: {Dept: "AI"}}
... )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
BDA_LAB>
```

LABORATORY PROGRAM – 2

Perform the following DB operations using Cassandra

Questions:

a) Create a keyspace by name Employee

b) Create a column family by name

- Employee-Info with attributes
- Emp_Id Primary Key, Emp_Name,
- Designation, Date_of_Joining,
- Salary, Dept_Name

c) Insert the values into the table in batch

d) Update Employee name and Department of Emp-Id 121

e) Sort the details of Employee records based on salary

f) Alter the schema of the table Employee_Info to add a column Projects which stores a set of Projects done by the corresponding Employee.

g) Update the altered table to add project names.

h) Create a TTL of 15 seconds to display the values of Employees.

COMMAND WITH OUTPUT

```
cqlsh> CREATE KEYSPACE Employee
   ... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};

cqlsh>
cqlsh> USE Employee;
cqlsh:employee> CREATE TABLE Employee_Info (
            ...     Emp_Id INT PRIMARY KEY,
            ...     Emp_Name TEXT,
            ...     Designation TEXT,
            ...     Date_of_Joining DATE,
            ...     Salary INT,
            ...     Dept_Name TEXT
            ... );
cqlsh:employee> BEGIN BATCH
            ... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
            ... VALUES (121, 'Manoj', 'Engineer', '2022-01-10', 50000, 'CSE');
            ...
            ... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
            ... VALUES (122, 'Rahul', 'Manager', '2021-05-15', 70000, 'ISE');
            ...
            ... INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
            ... VALUES (123, 'Anjali', 'Analyst', '2023-03-20', 45000, 'ECE');
            ... APPLY BATCH;
cqlsh:employee> UPDATE Employee_Info
            ... SET Emp_Name = 'Manoj Kumar', Dept_Name = 'AI'
            ... WHERE Emp_Id = 121;
cqlsh:employee> SELECT * FROM Employee_Info ORDER BY Salary DESC ALLOW FILTERING;
```

```
cqlsh:employee> ALTER TABLE Employee_Info
            ... ADD Projects SET<TEXT>;
cqlsh:employee> UPDATE Employee_Info
            ... SET Projects = {'Hadoop', 'Spark'}
            ... WHERE Emp_Id = 121;
cqlsh:employee>
cqlsh:employee> UPDATE Employee_Info
            ... SET Projects = {'ML', 'Data Mining'}
            ... WHERE Emp_Id = 122;
cqlsh:employee>
```

```
cqlsh:employee> INSERT INTO Employee_Info (Emp_Id, Emp_Name, Designation, Date_of_Joining, Salary, Dept_Name)
            ... VALUES (130, 'TempUser', 'Intern', '2024-01-01', 20000, 'CSE')
            ... USING TTL 15;
cqlsh:employee> SELECT * FROM Employee_Info;

 emp_id | date_of_joining | dept_name | designation | emp_name    | projects                | salary
--------+-----------------+-----------+-------------+-------------+-------------------------+--------
    123 |      2023-03-20 |       ECE |     Analyst |      Anjali |                    null |  45000
    130 |      2024-01-01 |       CSE |      Intern |    TempUser |                    null |  20000
    122 |      2021-05-15 |       ISE |     Manager |       Rahul | {'Data Mining', 'ML'} |  70000
    121 |      2022-01-10 |        AI |    Engineer | Manoj Kumar |   {'Hadoop', 'Spark'} |  50000

(4 rows)
cqlsh:employee> SELECT * FROM Employee_Info;

 emp_id | date_of_joining | dept_name | designation | emp_name    | projects                | salary
--------+-----------------+-----------+-------------+-------------+-------------------------+--------
    123 |      2023-03-20 |       ECE |     Analyst |      Anjali |                    null |  45000
    130 |      2024-01-01 |       CSE |      Intern |    TempUser |                    null |  20000
    122 |      2021-05-15 |       ISE |     Manager |       Rahul | {'Data Mining', 'ML'} |  70000
    121 |      2022-01-10 |        AI |    Engineer | Manoj Kumar |   {'Hadoop', 'Spark'} |  50000

(4 rows)
```

LABORATORY PROGRAM – 3

Perform the following DB operations using Cassandra

Questions:

a) Create a keyspace by name Library
b) Create a column family by name Library-Info with attributes

- Stud_Id Primary Key,
- Counter_value of type Counter,
- Stud_Name, Book-Name, Book-Id,
- Date_of_issue

c) Insert the values into the table in batch
d) Display the details of the table created and increase the value of the counter
e) Write a query to show that a student with id 112 has taken a book "BDA" 2 times.
f) Export the created column to a csv file
g) Import a given csv dataset from local file system into Cassandra column family

COMMAND WITH OUTPUT

```
cqlsh> CREATE KEYSPACE Library
   ... WITH replication = {'class': 'SimpleStrategy', 'replication_factor': 1};
cqlsh> USE Library;
cqlsh:library> CREATE TABLE Library_Info (
           ...     Stud_Id INT PRIMARY KEY,
           ...     Stud_Name TEXT,
           ...     Book_Name TEXT,
           ...     Book_Id INT,
           ...     Date_of_issue DATE
           ... );
cqlsh:library> CREATE TABLE Book_Counter (
           ...     Stud_Id INT,
           ...     Book_Name TEXT,
           ...     Counter_Value COUNTER,
           ...     PRIMARY KEY (Stud_Id, Book_Name)
           ... );
cqlsh:library> BEGIN BATCH
           ... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue)
           ... VALUES (111, 'Manoj', 'BDA', 201, '2024-04-01');
           ...
           ... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue)
           ... VALUES (112, 'Rahul', 'BDA', 202, '2024-04-02');
           ...
           ... INSERT INTO Library_Info (Stud_Id, Stud_Name, Book_Name, Book_Id, Date_of_issue)
           ... VALUES (113, 'Anjali', 'DBMS', 203, '2024-04-03');
           ... APPLY BATCH;
cqlsh:library> UPDATE Book_Counter
           ... SET Counter_Value = Counter_Value + 1
           ... WHERE Stud_Id = 112 AND Book_Name = 'BDA';
cqlsh:library>
cqlsh:library> UPDATE Book_Counter
           ... SET Counter_Value = Counter_Value + 1
           ... WHERE Stud_Id = 112 AND Book_Name = 'BDA';
cqlsh:library>
```

```
cqlsh:library> SELECT * FROM Library_Info;

 stud_id | book_id | book_name | date_of_issue | stud_name
---------+---------+-----------+---------------+----------
     111 |     201 |       BDA |    2024-04-01 |     Manoj
     113 |     203 |      DBMS |    2024-04-03 |    Anjali
     112 |     202 |       BDA |    2024-04-02 |     Rahul

(3 rows)
cqlsh:library> SELECT * FROM Book_Counter;

 stud_id | book_name | counter_value
---------+-----------+---------------
     112 |       BDA |             2

(1 rows)
cqlsh:library> SELECT * FROM Book_Counter
           ... WHERE Stud_Id = 112 AND Book_Name = 'BDA';

 stud_id | book_name | counter_value
---------+-----------+---------------
     112 |       BDA |             2

(1 rows)
cqlsh:library> cqlsh -e "COPY Library.Library_Info TO 'library.csv' WITH HEADER = TRUE;"
           ...
cqlsh:library>
```

LABORATORY PROGRAM – 4

Execution of HDFS Commands for interaction with Hadoop Environment. (Minimum 10 commands to be executed)

COMMAND WITH OUTPUT

```
Starting namenodes on [localhost]
Starting datanodes
```

```
Starting resourcemanager
Starting nodemanagers
```

```
~/Downloads$ hdfs dfs -mkdir /input_file
~/Downloads$ echo "Hello world!" > input.txt
~/Downloads$ hdfs dfs -put input.txt /input_file
~/Downloads$ hdfs dfs -ls /input_file

group          13 2026-05-05 00:29 /input_file/input.txt
~/Downloads$ hdfs dfs -cat /input_file/input.txt

~/Downloads$ hdfs dfs -rm -r /input_file/input.txt

~/Downloads$ hdfs dfs -ls /input_file
~/Downloads$ echo "Hello\nWorld" > input1.txt
~/Downloads$ hdfs dfs -cat /input1.txt
ile or directory
~/Downloads$ hdfs dfs -put input1.txt /input_file
~/Downloads$ hdfs dfs -cat /input_file/input1.txt

~/Downloads$ hdfs dfs -rm -r /input_file

~/Downloads$
```

LABORATORY PROGRAM – 5
Implement Wordcount program on Hadoop framework

```
import java.io.Exception
importorg.apache.hadoop.conf.Configured
; import org.apache.hadoop.fs.Path;
importorg.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
importorg.apache.hadoop.mapred.FileInputFormat;
importorg.apache.hadoop.mapred.FileOutputFormat;
import org.apache.hadoop.mapred.JobClient;
importorg.apache.hadoop.mapred.JobConf;
import org.apache.hadoop.util.Tool;
import org.apache.hadoop.util.ToolRunner;

public class WCDriver extends Configured implements Tool

  { public int run(String[] args) throws IOException {

    if (args.length < 2) {
      System.out.println("Please give valid inputs");
      return -1;
    }

    JobConf conf = new JobConf(WCDriver.class);
    conf.setJobName("WordCount");

    FileInputFormat.setInputPaths(conf, new Path(args[0]));
    FileOutputFormat.setOutputPath(conf, new
    Path(args[1]));

    conf.setMapperClass(WCMapper.clas
    s);
    conf.setReducerClass(WCReducer.cla
    ss);

    conf.setMapOutputKeyClass(Text.class);
    conf.setMapOutputValueClass(IntWritable.class
    );

    conf.setOutputKeyClass(Text.class);
    conf.setOutputValueClass(IntWritable.class);

    JobClient.runJob(conf);
```

```
    return 0;
}

// Main Method
```

```
    public static void main(String[] args) throws
      Exception { int exitCode = ToolRunner.run(new
      WCDriver(), args); System.out.println("Job Exit
      Code: " + exitCode);
    }
  }}
```

Mapper Code:

```
// Importing libraries
import
java.io.IOException;

import org.apache.hadoop.io.IntWritable;
import
org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapred.MapReduceBase;
import org.apache.hadoop.mapred.Mapper;
import
org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reporter;

public class WCMapper extends MapReduceBase
    implements Mapper<LongWritable, Text, Text, IntWritable> {

  // Map function
  public void map(LongWritable key, Text value,
            OutputCollector<Text, IntWritable>
            output, Reporter reporter) throws
            IOException {

    String line = value.toString();

    // Splitting the line on whitespace
    for (String word : line.split("\\s+"))
    {
      if (word.length() > 0) {
        output.collect(new Text(word), new IntWritable(1));
      }
    }
  }
}
```

Reducer Code

```
// Importing libraries
import
java.io.IOException;
import java.util.Iterator;
```

```
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapred.MapReduceBase;
import  org.apache.hadoop.mapred.OutputCollector;
import org.apache.hadoop.mapred.Reducer;
import org.apache.hadoop.mapred.Reporter;
```

```
public class WCReducer extends MapReduceBase
        implements Reducer<Text, IntWritable, Text, IntWritable> {

    // Reduce function
    public void reduce(Text key, Iterator<IntWritable> values,
                       OutputCollector<Text, IntWritable> output,
                       Reporter reporter) throws IOException {

        int count = 0;

        // Counting the frequency of each word
        while (values.hasNext()) {
            count += values.next().get();
        }

        output.collect(key, new IntWritable(count));
    }
}
```

```
2026-05-05 00:41:05,096 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-05-05 00:41:06,216 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute your application with ToolRunner to remedy this.
2026-05-05 00:41:06,257 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/yatharth/.staging/job_1777921102769_0001
2026-05-05 00:41:06,726 INFO input.FileInputFormat: Total input files to process : 1
2026-05-05 00:41:07,272 INFO mapreduce.JobSubmitter: number of splits:1
2026-05-05 00:41:07,700 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777921102769_0001
2026-05-05 00:41:07,700 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-05-05 00:41:08,026 INFO conf.Configuration: resource-types.xml not found
2026-05-05 00:41:08,031 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-05-05 00:41:10,366 INFO impl.YarnClientImpl: Submitted application application_1777921102769_0001
2026-05-05 00:41:10,557 INFO mapreduce.Job: The url to track the job: http://yatharth-VirtualBox:8088/proxy/application_1777921102769_0001/
2026-05-05 00:41:10,566 INFO mapreduce.Job: Running job: job_1777921102769_0001
2026-05-05 00:41:28,454 INFO mapreduce.Job: Job job_1777921102769_0001 running in uber mode : false
2026-05-05 00:41:28,466 INFO mapreduce.Job:  map 0% reduce 0%
2026-05-05 00:41:42,061 INFO mapreduce.Job:  map 100% reduce 0%
2026-05-05 00:41:48,139 INFO mapreduce.Job:  map 100% reduce 100%
2026-05-05 00:41:49,164 INFO mapreduce.Job: Job job_1777921102769_0001 completed successfully
2026-05-05 00:41:49,317 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=126
                FILE: Number of bytes written=551777
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=163
                HDFS: Number of bytes written=76
```

```
=       1
analysis.       1
bda     1
bda.    1
big     1
data    1
hadoop  1
hello   2
in      1
is      1
this    1
```

LABORATORY PROGRAM – 6

Implement Weather program on Hadoop framework

Questions:

From the following link extract the weather data
https://github.com/tomwhite/hadoopbook/tree/master/input/ncdc/all

a) Create a MapReduce program to find average temperature for each year from NCDC data set.

b) find the mean max temperature for every month.

CODE, COMMAND WITH OUTPUT – A
Driver Code

```
package temp;

import
org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import
org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class AverageDriver {
   public static void main(String[] args) throws Exception {
      if (args.length != 2) {
         System.err.println("Please enter both input and output
         parameters."); System.exit(-1);
      }

      // Creating a configuration and job instance
      Configuration conf = new Configuration();
      Job job = Job.getInstance(conf, "Average Calculation");
      job.setJarByClass(AverageDriver.class);

      // Input and output paths
      FileInputFormat.addInputPath(job, new Path(args[0]));
      FileOutputFormat.setOutputPath(job, new
      Path(args[1]));

      // Setting mapper and reducer classes
```

```
job.setMapperClass(AverageMapper.class);
job.setReducerClass(AverageReducer.class)
;
```

```
        // Output key and value types
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class)
        ;

        // Submitting the job and waiting for it to complete
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

MAPPER CODE

```
package temp;


import java.io.IOException;


import org.apache.hadoop.io.IntWritable;
import
org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;


public class AverageMapper extends Mapper<LongWritable, Text, Text, IntWritable> {


    public static final int MISSING = 9999;


    @Override
    public void map(LongWritable key, Text value, Context context)
            throws IOException, InterruptedException {


        String line = value.toString();


        // Extract year from fixed position
        String year = line.substring(15, 19);
        int temperature;
```

```
// Determine if there's a '+' sign
if (line.charAt(87) == '+') {
```

```
            temperature = Integer.parseInt(line.substring(88, 92));
        } else {
            temperature = Integer.parseInt(line.substring(87, 92));
        }

        // Quality check character
        String quality = line.substring(92, 93);

        // Only emit if data is valid
        if (temperature != MISSING && quality.matches("[01459]")) {
            context.write(new Text(year), new IntWritable(temperature));
        }
    }
```

REDUCER CODE

```
package temp;

import java.io.IOException;

import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class AverageReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    public void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        int sumTemp = 0;
        int count = 0;
```

```
        for (IntWritable value : values) {
            sumTemp += value.get();
            count++;
        }

        if (count > 0) {
            int average = sumTemp / count;
            context.write(key, new IntWritable(average));
        }
    }
```

```
2026-05-05 01:09:16,335 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-05-05 01:09:17,266 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interf
2026-05-05 01:09:17,346 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/yatharth/.stagi
2026-05-05 01:09:17,825 INFO input.FileInputFormat: Total input files to process : 1
2026-05-05 01:09:18,452 INFO mapreduce.JobSubmitter: number of splits:1
2026-05-05 01:09:18,799 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777921102769_0002
2026-05-05 01:09:18,800 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-05-05 01:09:19,036 INFO conf.Configuration: resource-types.xml not found
2026-05-05 01:09:19,037 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-05-05 01:09:19,955 INFO impl.YarnClientImpl: Submitted application application_1777921102769_0002
2026-05-05 01:09:20,038 INFO mapreduce.Job: The url to track the job: http://yatharth-VirtualBox:8088/proxy/application_1777921102769_0
2026-05-05 01:09:20,039 INFO mapreduce.Job: Running job: job_1777921102769_0002
2026-05-05 01:09:31,711 INFO mapreduce.Job: Job job_1777921102769_0002 running in uber mode : false
2026-05-05 01:09:31,712 INFO mapreduce.Job:  map 0% reduce 0%
2026-05-05 01:09:37,911 INFO mapreduce.Job:  map 100% reduce 0%
2026-05-05 01:09:43,988 INFO mapreduce.Job:  map 100% reduce 100%
2026-05-05 01:09:44,021 INFO mapreduce.Job: Job job_1777921102769_0002 completed successfully
2026-05-05 01:09:44,147 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=71
                FILE: Number of bytes written=551351
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=145
                HDFS: Number of bytes written=24
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
                HDFS: Number of bytes read erasure-coded=0
```

LABORATORY PROGRAM – 7

For a given Text file, Create a Map Reduce program to sort the content in an alphabetic order listing only top 10 maximum occurrences of words.

```
import
org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;

import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class TopNDriver {
   public static void main(String[] args) throws Exception {
      if (args.length != 3) {
         System.err.println("Usage: TopNDriver <in> <temp-out> <final-out>");
         System.exit(2);
      }

      Configuration conf = new Configuration();

      // === Job 1: Word Count ===
      Job wcJob = Job.getInstance(conf, "word count");
      wcJob.setJarByClass(TopNDriver.class);
      wcJob.setMapperClass(WordCountMapper.class);
      wcJob.setCombinerClass(WordCountReducer.class)
      ;
      wcJob.setReducerClass(WordCountReducer.class);
      wcJob.setOutputKeyClass(Text.class);
      wcJob.setOutputValueClass(IntWritable.class);

      FileInputFormat.addInputPath(wcJob, new Path(args[0]));
      Path tempDir = new Path(args[1]);
      FileOutputFormat.setOutputPath(wcJob, tempDir);

      if (!wcJob.waitForCompletion(true))
         { System.exit(1);
      }
```

// === Job 2: Top N ===

```
        Job topJob = Job.getInstance(conf, "top 10 words");
        topJob.setJarByClass(TopNDriver.class);
        topJob.setMapperClass(TopNMapper.class);
        topJob.setReducerClass(TopNReducer.class);
        topJob.setMapOutputKeyClass(IntWritable.class);
        topJob.setMapOutputValueClass(Text.class);
        topJob.setOutputKeyClass(Text.class);
        topJob.setOutputValueClass(IntWritable.class);

        FileInputFormat.addInputPath(topJob, tempDir);
        FileOutputFormat.setOutputPath(topJob, new
        Path(args[2]));

        System.exit(topJob.waitForCompletion(true) ? 0 : 1);
    }
}
```

Mapper Code (WordCountMapper.java)

```
package samples.topn;

import java.io.IOException;
import
java.util.StringTokenizer;

import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
public class WordCountMapper extends Mapper<Object, Text, Text, IntWritable> {
    private static final IntWritable ONE = new IntWritable(1);
    private Text word = new Text();

    // characters to normalize into spaces
    private String tokens = "[_|$#<>\\^=\\[\\]\\*/\\\\,;,.\\-:()?!\"']";

    @Override
    protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

        // clean & tokenize
        String clean = value.toString()
                .toLowerCase()
                .replaceAll(tokens, " ");
```

```
StringTokenizer itr = new StringTokenizer(clean);
```

```
        while (itr.hasMoreTokens()) {
            word.set(itr.nextToken().trim())
            ; context.write(word, ONE);
        }
    }
}
```

Mapper Code (TopNMapper.java)

```
package samples.topn;

import

java.io.IOException;

import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;
public class TopNMapper extends Mapper<Object, Text, IntWritable, Text> {
    private IntWritable count = new IntWritable();
    private Text word = new Text();

    @Override
    protected void map(Object key, Text value, Context context)
            throws IOException, InterruptedException {

        // input line: word \t count
        String[] parts = value.toString().split("\\t");

        if (parts.length == 2) {
            word.set(parts[0]);
            count.set(Integer.parseInt(parts[1])
            );

            // emit count → word, so Hadoop sorts by count
            context.write(count, word);
        }
    }
}
```

Reducer Code (WordCountReducer.java)

```
package samples.topn;

import
```

```
java.io.IOException;

import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;
```

```
public class WordCountReducer extends Reducer<Text, IntWritable, Text, IntWritable> {

    @Override
    protected void reduce(Text key, Iterable<IntWritable> values, Context context)
            throws IOException, InterruptedException {

        int sum = 0;

        for (IntWritable val : values) {
            sum += val.get();
        }

        context.write(key, new IntWritable(sum));
    }
}
```

Reducer Code (TopNReducer.java)

```
package samples.topn;

import
java.io.IOException;
import java.util.ArrayList;
import
java.util.Collections;
import java.util.List;
import java.util.Map;
import
java.util.TreeMap;

import
org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

public class TopNReducer extends Reducer<IntWritable, Text, Text, IntWritable> {

    // TreeMap with descending order of keys (counts)
    private TreeMap<Integer, List<String>> countMap =
            new TreeMap<>(Collections.reverseOrder());

    @Override
    protected void reduce(IntWritable key, Iterable<Text> values, Context context)
            throws IOException, InterruptedException {
```

```
int cnt = key.get();

List<String> words = countMap.getOrDefault(cnt, new ArrayList<>());
```

```
        for (Text w : values) {
            words.add(w.toString())
            ;
        }

        countMap.put(cnt, words);
    }

    @Override
    protected void cleanup(Context context)
            throws IOException, InterruptedException {

        // collect top 10 word→count pairs
        List<WordCount> topList = new ArrayList<>();
        int seen = 0;

        for (Map.Entry<Integer, List<String>> entry : countMap.entrySet()) {
            int cnt = entry.getKey();

            for (String w : entry.getValue()) {
                topList.add(new WordCount(w,
                cnt)); seen++;

                if (seen == 10) break;
            }

            if (seen == 10) break;
        }

        // emit results
        for (WordCount wc : topList) {
            context.write(new Text(wc.getWord()), new IntWritable(wc.getCount()));
        }
    }
}
        // sort these 10 entries alphabetically by word
        Collections.sort(topList, (a, b) -> a.word.compareTo(b.word));

        // emit final top 10 in alphabetical order
        for (WordCount wc : topList) {
            context.write(new Text(wc.word), new IntWritable(wc.count));
        }
    }
```

// helper class

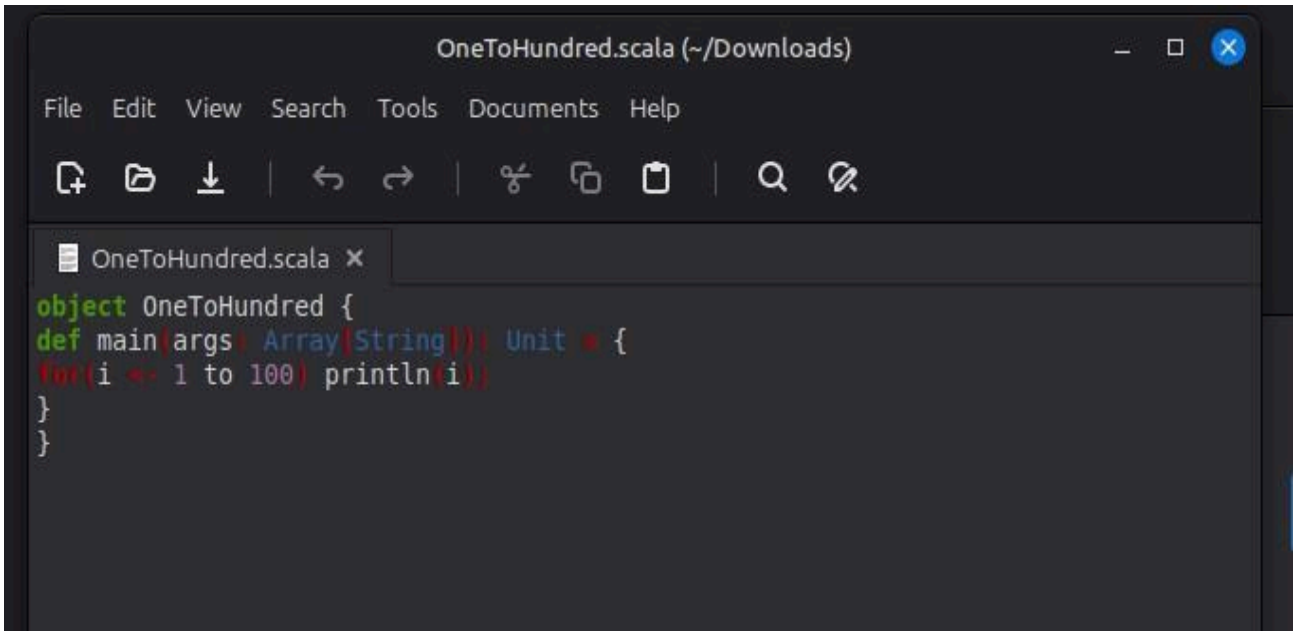
```
    private static class WordCount {
        String word;
        int count;

        WordCount(String w, int c) {
            word = w;
            count = c;
        }
    }
}
```

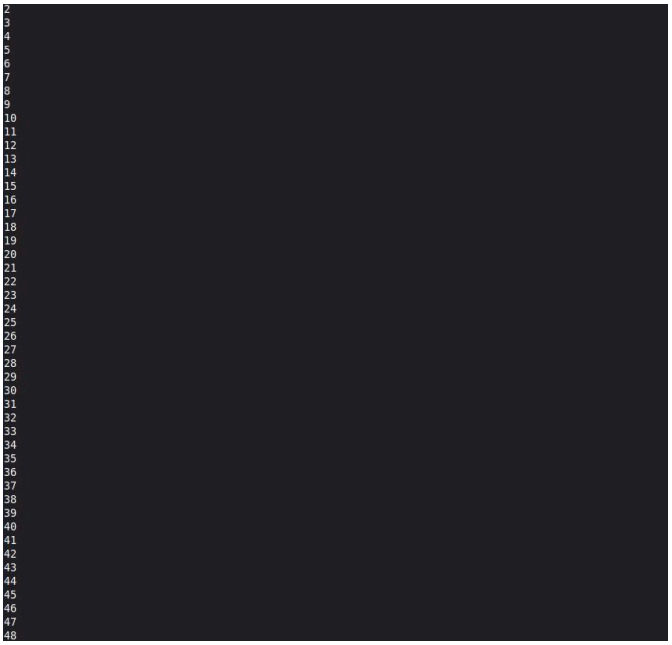
```
2026-05-05 01:26:39,747 INFO client.DefaultNoHARMFailoverProxyProvider: Connecting to ResourceManager at /0.0.0.0:8032
2026-05-05 01:26:40,113 WARN mapreduce.JobResourceUploader: Hadoop command-line option parsing not performed. Implement the Tool interface and execute you
2026-05-05 01:26:40,150 INFO mapreduce.JobResourceUploader: Disabling Erasure Coding for path: /tmp/hadoop-yarn/staging/yatharth/.staging/job_1777921102769
2026-05-05 01:26:40,491 INFO input.FileInputFormat: Total input files to process : 1
2026-05-05 01:26:40,621 INFO mapreduce.JobSubmitter: number of splits:1
2026-05-05 01:26:40,822 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777921102769_0003
2026-05-05 01:26:40,822 INFO mapreduce.JobSubmitter: Executing with tokens: []
2026-05-05 01:26:40,993 INFO conf.Configuration: resource-types.xml not found
2026-05-05 01:26:40,994 INFO resource.ResourceUtils: Unable to find 'resource-types.xml'.
2026-05-05 01:26:41,110 INFO impl.YarnClientImpl: Submitted application application_1777921102769_0003
2026-05-05 01:26:41,189 INFO mapreduce.Job: The url to track the job: http://yatharth-VirtualBox:8088/proxy/application_1777921102769_0003/
2026-05-05 01:26:41,190 INFO mapreduce.Job: Running job: job_1777921102769_0003
2026-05-05 01:26:49,456 INFO mapreduce.Job: Job job_1777921102769_0003 running in uber mode : false
2026-05-05 01:26:49,458 INFO mapreduce.Job:  map 0% reduce 0%
2026-05-05 01:26:54,562 INFO mapreduce.Job:  map 100% reduce 0%
2026-05-05 01:27:01,679 INFO mapreduce.Job:  map 100% reduce 100%
2026-05-05 01:27:01,697 INFO mapreduce.Job: Job job_1777921102769_0003 completed successfully
2026-05-05 01:27:01,850 INFO mapreduce.Job: Counters: 54
        File System Counters
                FILE: Number of bytes read=325
                FILE: Number of bytes written=551835
                FILE: Number of read operations=0
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=260
                HDFS: Number of bytes written=41
                HDFS: Number of read operations=8
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
```

LABORATORY PROGRAM – 8

Write a Scala program to print numbers from 1 to 100 using for loop.

CODE, COMMAND WITH OUTPUT





LABORATORY PROGRAM – 9

Using RDD and FlatMap count how many times each word appears in a file and write out a list of words whose count is strictly greater than 4 using Spark.

CODE, COMMAND WITH OUTPUT

```
>>> rdd = spark.sparkContext.textFile("input.txt")
>>> words = rdd.flatMap(lambda line: line.split(" "))
>>> pairs = words.map(lambda word: (word, 1))
>>> counts = pairs.reduceByKey(lambda a, b: a + b)
>>> result = counts.filter(lambda x: x[1] > 4)
>>> result.collect()
[('hello', 5), ('spark', 6)]
```

LABORATORY PROGRAM – 10

Write a simple streaming program in Spark to receive text data streams on a particular port, perform basic text cleaning (like white space removal, stop words removal, lemmatization, etc.), and print the cleaned text on the screen. (Open Ended Question).

CODE, COMMAND WITH OUTPUT

```
# Install NLTK and download required data (run once)
!pip install nltk

import nltk
nltk.download('punkt')
nltk.download('stopwords'
)
nltk.download('wordnet')

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, lower, regexp_replace, split, explode, udf
from pyspark.sql.types import ArrayType, StringType
from pyspark.ml.feature import StopWordsRemover
from nltk.stem import WordNetLemmatizer

# Initialize SparkSession
spark = SparkSession.builder.appName("TextProcessing").getOrCreate()

# Define your input lines
lines = [
    "Hello, I hate you.",
    "I hate that I love you.",
    "Don't want to, but I can't
    put", "nobody else above
    you."
]

# Create DataFrame from lines
df = spark.createDataFrame(lines, "string").toDF("value")

# Step 1: Lowercase and remove punctuation
df_clean = df.select(
    regexp_replace(lower(col("value")), "[^a-zA-Z\\s]", "").alias("cleaned")
```

)

```
# Step 2: Tokenize the cleaned text
df_tokens = df_clean.select(
    split(col("cleaned"), "\\s+").alias("tokens")
)

# Step 3: Remove stop words
remover = StopWordsRemover(inputCol="tokens", outputCol="filtered")
df_filtered = remover.transform(df_tokens)

# Step 4: Lemmatization using NLTK WordNetLemmatizer with UDF
lemmatizer = WordNetLemmatizer()

def lemmatize_words(words):
    return [lemmatizer.lemmatize(word) for word in words]
lemmatize_udf = udf(lemmatize_words, ArrayType(StringType()))
df_lemmatized = df_filtered.withColumn(
    "lemmatized",
    lemmatize_udf(col("filtered")
    )
)

# Step 5: Explode the lemmatized words and show results
df_lemmatized.select(
    explode(col("lemmatized")).alias("word")
).show(truncate=False)
```